PLAYWRIGHT AUTOMATION FRAMEWORK

From-Scratch Implementation Guide

Step-by-step implementation of a Playwright TypeScript automation framework

1. Framework Objective

Build a Playwright + TypeScript automation framework from an empty folder. Implement each layer in sequence, understand its responsibility, and validate how the components work together.

- Follow the numbered build sequence. Complete and validate each milestone before moving to the next.
- Treat code examples as implementation patterns. Match imports, method names, and paths consistently when adapting them.
- The framework targets Salesforce UI flows and Salesforce lead APIs. Use a dedicated test account and test environment.
- Do not commit credentials, access tokens, cookies, storage-state files, or environment secrets to source control.
- Apply the security and maintainability practices described in each implementation section.

2. Framework Components and Responsibilities

Area / path	Responsibility	How to recreate it
playwright.config.ts	Central test runner configuration: test directory, timeout, browser projects, retries, reporter, trace, video and permissions.	Create early; keep environment-specific settings configurable.
pages/	Salesforce page objects such as login, home, account, lead and mobile publisher; selectors are centralized in selectors.ts.	Build page objects after the wrapper and constants.
helpers/playwright.ts	Abstract PlaywrightWrapper containing reusable browser interactions and helper methods.	Implement common actions once; page objects inherit from the wrapper.
customFixtures/salesForceFixture.ts	Extends Playwright Test with typed page-object fixtures.	Register each page object through test.extend and consume it in specs.
tests/	UI test specifications for account, lead, contact, mobile publisher and opportunity scenarios.	Keep tests focused on scenario orchestration and assertions.
data/	CSV/JSON test data and TypeScript interfaces.	Separate test inputs from test logic; validate data shapes.
helpers/	CSV, JSON, Excel, Faker, DB, request, verification and JSON-reading utilities.	Add utilities only when a real use case requires them.
api/api_services/	API service layer, including token generation and lead operations.	Keep endpoint calls and payload handling outside test specifications.
api/api_Tests/	API test cases for create, delete and update lead workflows.	Use API fixtures or request context, and assert status and response body.
constants/	URL and credential mappings.	Keep non-secret constants here; move secrets to environment variables.
logins/	Location referenced for saved Playwright storage state.	Treat generated state as a secret; ignore it in Git.
.github/workflows/playwright.yml	GitHub Actions workflow to install dependencies and browsers, run tests and upload report artifacts.	Add after local test execution is reliable.
package.json / tsconfig.json	Dependency manifest, scripts and TypeScript compiler settings.	Create at project initialization and maintain reproducible dependency installs.

3. Target Architecture and Execution Flow

```
Test specification
 └── Custom fixture (salesForceFixture.ts)
 └── Page object (pages/salesforceAccountPage.ts, etc.)
 └── PlaywrightWrapper (helpers/playwright.ts)
 └── Playwright Page / BrowserContext
 └── Salesforce application

Test specification
 ├── Test data (CSV / JSON / interface / Faker)
 ├── Assertions and test annotations
 └── API test → API service → request / token helper → API endpoint

Playwright Test Runner
 ├── playwright.config.ts
 ├── HTML report / trace / video
 └── GitHub Actions workflow
```

Design principle: test files describe the business scenario; page objects describe application interactions; the wrapper contains shared low-level actions; fixtures construct and provide objects; helpers manage cross-cutting tasks; configuration controls execution.

4. Prerequisites and Project Initialization

Install a supported Node.js LTS release and VS Code. Verify the installation:

```
node --version
npm --version
```

Create the project and install Playwright Test:

```
mkdir SonicFramework
cd SonicFramework
npm init -y
npm install -D @playwright/test typescript @types/node
npx playwright install
```

Install only the supporting packages that the implementation actually uses:

```
npm install @faker-js/faker dotenv csv-parse csv-parser xlsx axios mysql2
npm install -D allure-commandline
npm install allure-playwright
```

The uploaded package.json uses Playwright Test, Faker, Allure, Axios, CSV parsers, dotenv, mysql2 and xlsx. Do not install duplicate libraries merely because they appear in a sample; choose one CSV approach and document it.

5. Create the Directory Structure

```
SonicFramework/
├── api/
│   ├── api_services/
│   └── api_Tests/
├── constants/
├── customFixtures/
├── data/
│   └── apiData/
├── e2e/
├── helpers/
├── logins/ # generated storage state; do not commit
├── pages/
├── tests/
├── playwright.config.ts
├── package.json
├── tsconfig.json
├── .env # local secrets; do not commit
└── .gitignore
```

Build order matters. Create configuration first, then the wrapper, page objects, fixtures, data utilities, tests, API layer, and CI pipeline.

6. Configure TypeScript and Playwright

A typical tsconfig.json for this style of project is:

```
{
 "compilerOptions": {
 "target": "ES2022",
 "module": "commonjs",
 "moduleResolution": "node",
 "strict": true,
 "esModuleInterop": true,
 "resolveJsonModule": true,
 "types": ["node", "@playwright/test"],
 "outDir": "dist"
 },
 "include": ["**/*.ts"],
 "exclude": ["node_modules", "dist"]
}
```

Configure the test runner to use ./tests, a suitable test timeout, a 5-second expect timeout, HTML reporting, trace/video capture, and Chromium. Enable headed mode locally and headless mode in CI. Add permissions only when the tests require them.

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
 testDir: './tests',
 timeout: 120_000,
 expect: { timeout: 5_000 },
 fullyParallel: true,
 forbidOnly: Boolean(process.env.CI),
 retries: process.env.CI ? 2 : 0,
 workers: process.env.CI ? 1 : undefined,
 reporter: [['html', { open: 'never' }]],
 use: {
 trace: 'on-first-retry',
 video: 'on',
 headless: Boolean(process.env.CI)
 },
 projects: [
 { name: 'chromium', use: { ...devices['Desktop Chrome'] } }
 ]
});
```

Configuration note: headed execution can be useful while learning, but CI usually requires headless execution. Permissions such as geolocation and notifications should be enabled only when tests need them. Avoid using long hard waits; rely on Playwright's locator auto-waiting and explicit state assertions.

7. Build the Reusable Playwright Wrapper

Create helpers/playwright.ts with an abstract PlaywrightWrapper. Pass Page and BrowserContext to its constructor and place reusable actions such as typing, clicking, navigation, text retrieval, and storage-state handling in this class. The wrapper centralizes repeated browser interactions and is not a concrete application page.

```
import { Page, BrowserContext, test } from '@playwright/test';

export abstract class PlaywrightWrapper {
 constructor(
 protected readonly page: Page,
 protected readonly context: BrowserContext
 ) {}

 async fill(locator: string, name: string, value: string): Promise<void> {
 await test.step(`Fill ${name}`, async () => {
 await this.page.locator(locator).fill(value);
 });
 }
```

```
async click(locator: string, description: string): Promise<void> {
await test.step(`Click ${description}`, async () => {
await this.page.locator(locator).click();
});
}

async loadApplication(url: string): Promise<void> {
await this.page.goto(url);
}

async saveStorageState(filePath: string): Promise<void> {
await this.context.storageState({ path: filePath });
}
}
```

- Use semantic names such as fill, click, selectOption, check, uploadFile and waitForVisible.
- Prefer locator-based auto-waiting instead of adding waitForTimeout or redundant waitForSelector calls before every action.
- Avoid logging passwords, access tokens or sensitive test data in test.step descriptions.
- Use explicit return types where they make the API clearer.
- Store newly opened tabs in per-test instance properties or provide them through fixtures. Avoid static page state because it can leak across tests.
- Catch errors only when you can add useful context and rethrow them. Do not swallow failures and let tests appear successful.

8. Build Constants and Selectors

Store URL constants under constants/ and element selectors under pages/selectors.ts. This reduces duplication and makes application changes easier to manage.

```
/* constants/urlConstants.ts */
export const URLConstants = {
 adminURL1: process.env.SALESFORCE_URL ?? ''
};

/* pages/selectors.ts */
export const selectors = {
 username: '#username',
 password: '#password',
 loginBtn: '#Login',
 applauncherIcon: '.slds-icon-waffle',
 homeLabel: "//h1//span[text()='Home']"
};
```

Prefer role-, label- and test-id-based locators when the application supports them. Keep selectors close to the page object or in a centralized selector module according to the project's convention. Do not store real passwords in credentialData.ts; use environment variables or a secrets manager.

9. Implement Page Object Model (POM)

Create page objects such as SalesforceLoginPage, SalesforceHomePage, SalesforceAccountPage, SalesforceLeadPage and SalesforceMobilePublisherPage. Each page object represents a user-facing page or application area and inherits common interactions from PlaywrightWrapper.

```
import { Page, BrowserContext, expect } from '@playwright/test';
import { PlaywrightWrapper } from '../helpers/playwright';
import { URLConstants } from '../constants/urlConstants';
import { selectors } from './selectors';

export class SalesforceLoginPage extends PlaywrightWrapper {
 constructor(page: Page, context: BrowserContext) {
 super(page, context);
 }

 async openLoginPage(): Promise<void> {
 await this.loadApplication(URLConstants.adminURL1);
 }

 async login(username: string, password: string): Promise<void> {
 await this.fill(selectors.username, 'username', username);
 await this.fill(selectors.password, 'password', password);
 await this.click(selectors.loginBtn, 'Login button');
 }

 async verifyHomePage(): Promise<void> {
 await expect(this.page.locator(selectors.homeLabel)).toHaveText('Home');
 }
}
```

Recommended implementation order:

1. Create selectors and URL configuration.
2. Create the login page object and prove that login works.
3. Create the home page object and navigation methods.
4. Create business page objects such as account and lead pages.
5. Move repeated interactions into the wrapper only when there is a clear shared use case.
6. Keep business workflow assertions in tests or page-level verification methods consistently; avoid duplicating the same assertion in both places without a reason.

10. Register Page Objects with Custom Fixtures

customFixtures/salesForceFixture.ts extends Playwright's base test and provides typed page-object fixtures. This lets a test request SalesforceLogin or SalesforceAccount directly instead of constructing objects in every test.

```
import { test as base } from '@playwright/test';
import { SalesforceLoginPage } from '../pages/salesforceLogin';
import { SalesforceAccountPage } from '../pages/salesforceAccountPage';

type FrameworkFixtures = {
 SalesforceLogin: SalesforceLoginPage;
 SalesforceAccount: SalesforceAccountPage;
};

export const test = base.extend<FrameworkFixtures>({
 SalesforceLogin: async ({ page, context }, use) => {
 await use(new SalesforceLoginPage(page, context));
 },
 SalesforceAccount: async ({ page, context }, use) => {
 await use(new SalesforceAccountPage(page, context));
 }
});

export { expect } from '@playwright/test';
```

- Add each new page object to the fixture type and provide its implementation in test.extend.
- Import test from the custom fixture module in tests that need these objects.
- Fixtures are created within Playwright's test lifecycle and can benefit from automatic cleanup of the underlying page/context.
- Keep fixture names and page-object names consistent; avoid unnecessary capitalization differences in local variables.

11. Add Data-Driven Testing

Create CSV and JSON test data, TypeScript interfaces, Faker utilities, and JSON/CSV/Excel helpers. Implement data management after one basic UI test passes, so failures are easier to isolate.

```
/* data/account.interface.ts */
export interface AccountData {
 Rating: string;
 Type: string;
```

```
 Industry: string;
 Ownership: string;
 BillingStreet: string;
 BillingCity: string;
 PostalCode: string;
 BillingState: string;
 BillingCountry: string;
}

/* Example Faker usage */
import { faker } from '@faker-js/faker';

export const FakerData = {
 getAccountName: () => faker.company.name(),
 getMobileNumber: () => faker.string.numeric(10)
};
```

- CSV is useful for tabular test inputs; JSON is useful for structured test data and small runtime records.
- TypeScript interfaces document the expected shape of data but do not validate untrusted runtime input by themselves.
- Faker values should be unique where the application requires unique names or identifiers.
- Avoid modifying a shared JSON test-data file during parallel test execution unless writes are isolated or synchronized.
- Resolve file paths consistently, preferably from process.cwd() or a well-defined project root; verify the path before running in CI.

12. Write the First UI Test

The account test can read CSV data, generate an account name, use JSON data where needed, navigate through Salesforce, fill the account form, and verify the created account. This workflow integrates data, fixtures, page objects, and assertions.

```
import { test, expect } from '../customFixtures/salesForceFixture';

test('Create an account', async ({ SalesforceLogin, SalesforceHome, SalesforceAccount }) => {
 await SalesforceLogin.openLoginPage();
 // Authenticate using a secure test credential strategy.
 // Use page-object methods to navigate to Accounts and create the record.
 // Assert the created record using a stable, observable UI state.
});
```

Build the scenario in small increments: login, verify login, navigate to Accounts, open the New form, populate required fields, save, and verify the created record. Only add CSV iteration after a single-record scenario is stable.

Storage-state note: the framework calls test.use({ storageState: './logins/salesforceLogin.json' }) in the account test, while its login page also has a method that saves state. Make sure the state file has been generated before dependent tests run. Never commit it

13. API Automation Layer

Organize API automation into a token-generation service, API service methods such as lead operations, and API test specifications for creating, deleting, and updating Salesforce leads.

```
/* Illustrative API test pattern */
import { test, expect } from '@playwright/test';

test('API returns an expected response', async ({ request }) => {
 const response = await request.get(process.env.API_HEALTH_URL ?? '');
 expect(response.ok()).toBeTruthy();
});
```

- Keep token acquisition, endpoint URLs, headers and payload construction out of the test specification.
- Assert HTTP status, relevant response fields and business outcomes, not only that a request completed.
- Do not hard-code access tokens or commit OAuth secrets. Load secrets through environment variables or CI secret storage.
- Use a dedicated test environment and clean up created data when appropriate.
- Use APIRequestContext or a service wrapper consistently, and document token expiry/refresh behaviour.

14. Helpers and Utility Modules

Helper	Role in this framework	Validation exercise
csvUtil.ts	Read tabular test data from CSV.	Load a sample CSV, assert row count and expected columns.
jsonReader.ts / jsonDataHandler.ts	Read and update JSON test data.	Read a known key, update a test key and confirm the new value.
fakerUtils.ts	Generate test data.	Generate several values and check format/uniqueness where required.
excelUtils.ts	Read or process spreadsheet data using xlsx.	Read a workbook sheet and verify expected rows/columns.
dbUtil.ts	Database access utilities using the configured DB client.	Run a safe read-only query against a test database and close the connection.
requestUtils.ts	Shared request functionality.	Validate headers, status handling and error propagation.
verificationUtils.ts	Reusable verification helpers.	Use a helper in a test and confirm a failed condition fails the test.
helpers/playwright.ts	Reusable browser interaction methods.	Unit-test or demonstrate key methods through a small browser scenario.

Avoid creating generic helper methods that hide meaningful assertions or business behaviour. Utilities should have a clear responsibility and predictable inputs/outputs.

15. Authentication, Environment Variables and Secrets

Store credentials and saved login-state data securely:

```
# .env.example – placeholders only
SALESFORCE_URL=https://your-test-instance.example
SALESFORCE_USERNAME=replace-with-test-user
SALESFORCE_PASSWORD=replace-with-secret

# .gitignore
node_modules/
playwright-report/
test-results/
allure-results/
allure-report/
.env
logins/*.json
```

- Keep .env out of Git and commit only .env.example with placeholder values.
- Use CI repository secrets for credentials in GitHub Actions.
- Do not place real credentials in test annotations, console output, screenshots or report attachments.
- Treat browser storage state as equivalent to a session credential and protect it accordingly.
- If credentials or tokens were ever committed, rotate them; deleting the text in a later commit is not sufficient.

16. Reporting, Traces, Video and Debugging

Configure HTML reporting, video recording, and trace capture on the first retry. If Allure reporting is required, configure its reporter explicitly; installing dependencies alone does not enable it.

```
# Run tests
npx playwright test

# Run a particular test file
npx playwright test tests/TC001_create_account.spec.ts

# Run in headed mode
npx playwright test --headed

# Run with Playwright Inspector
npx playwright test --debug

# Open the generated HTML report
npx playwright show-report
```

If Allure is required, explicitly configure the reporter, generate the report and publish its artifacts. Keep Playwright HTML reporting and Allure reporting as distinct choices unless both are intentionally configured.

17. GitHub Actions CI/CD

Configure .github/workflows/playwright.yml to run on pushes and pull requests to main/master, check out the repository, set up Node.js, install dependencies and browsers, run Playwright tests, and upload the HTML report.

```
name: Playwright Tests
on:
 push:
 branches: [main, master]
 pull_request:
 branches: [main, master]

jobs:
 test:
 runs-on: ubuntu-latest
 timeout-minutes: 60
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-node@v4
 with:
 node-version: 20
 cache: npm
 - name: Install dependencies
 run: npm ci
 - name: Install browsers
 run: npx playwright install --with-deps
 - name: Run tests
 run: npx playwright test
 - uses: actions/upload-artifact@v4
 if: always()
 with:
 name: playwright-report
 path: playwright-report/
 retention-days: 30
```

npm ci requires a committed package-lock.json consistent with package.json. Generate and commit package-lock.json before using npm ci in a clean CI runner. Configure required secrets and the correct test URL in repository settings. Do not assume UI tests can run successfully without access to the Salesforce test environment.

18. Recommended Build Milestones

Milestone	Deliverable	Pass condition
1. Bootstrap	Node, npm, package.json, TypeScript and Playwright installed.	A sample Playwright test runs.
2. Configuration	playwright.config.ts and tsconfig.json.	Correct test discovery and browser launch.
3. Wrapper	Reusable interaction class.	A test can navigate, fill and click without duplicated mechanics.
4. POM	Login and one business page object.	One end-to-end workflow works using page objects.

5. Fixtures	Typed custom fixtures.	Tests receive page objects through fixtures.
6. Data utilities	CSV/JSON/Faker data integration.	The test creates a record from external data.
7. UI suite	Account, lead, contact and other UI tests.	Independent tests are stable and assertions verify outcomes.
8. API suite	Token and lead service layers plus API specs.	Create/update/delete scenarios validate status and response data.
9. Reporting	HTML report, trace and video.	Failure evidence can be opened and understood.
10. CI/CD	GitHub Actions workflow and artifacts.	A clean CI runner installs dependencies and executes tests.

Do not proceed to the next milestone while the previous milestone has unresolved setup or compilation failures.

19. Framework Quality and Reliability

- The configuration currently uses fullyParallel: true. Review shared Salesforce records and JSON file writes before enabling parallel execution; use isolated data and unique records.
- The login test and storage-state reuse need a deterministic setup sequence. A test should not depend on a state file that another test may or may not have generated first.
- The wrapper contains a static newPage property. Static browser-page state can cause cross-test coupling; use per-test state or fixture-managed pages.
- Some wrapper methods use forced actions and explicit waits. Prefer normal locator actions and assertions; use force only when there is a documented reason.
- The source imports expect from 'playwright/test' in one page class while the package uses @playwright/test. Standardize imports on @playwright/test unless there is a deliberate reason otherwise.
- The account test updates JSON data while iterating CSV rows. Parallel or overlapping executions can overwrite shared data; use unique per-test output or avoid runtime mutation.
- The package includes Allure dependencies, but the shown config uses the HTML reporter. Configure Allure explicitly if it is part of the target deliverable.
- The GitHub Actions workflow uses npm ci, so a matching package-lock.json is required.
- Review all API and DB utilities for connection cleanup, error handling, timeouts and secret management.
- Validate the current code before treating optional README topics such as Jira integration or global setup as implemented features.

20. Final Verification Checklist

☐A clean checkout can install dependencies successfully.

☐Playwright browsers install successfully on a new machine.

☐TypeScript compiles without errors.

☐The configured test directory discovers the expected tests.

☐Login and storage-state setup work deterministically.

☐Page objects contain application interactions and tests contain clear scenario flow.

☐Custom fixtures create the expected page objects.

☐Data utilities correctly read and validate CSV/JSON inputs.

☐Generated data avoids collisions across tests.

☐UI tests assert observable business outcomes.

☐API tests validate status codes and response bodies.

☐Failed tests retain useful traces, videos or reports without exposing secrets.

☐Secrets and storage-state files are excluded from version control.

☐The CI workflow has a lock file, valid secrets and report artifacts.

☐The learner can explain the architecture and recreate each component without copying the finished repository.

21. Interview Explanation Template

"I built a modular automation framework using Playwright with TypeScript. The test specifications describe business scenarios, while page objects encapsulate Salesforce page interactions. A shared abstract wrapper provides reusable browser operations, and custom Playwright fixtures construct page objects and inject them into tests. Test data is separated into CSV and JSON files, with TypeScript interfaces and Faker utilities supporting data-driven scenarios. The framework also separates API service logic from API tests and includes supporting helpers for requests, verification, spreadsheet and database operations. Playwright configuration controls browser projects, retries, timeouts and diagnostic artifacts. GitHub Actions installs dependencies and browsers, executes tests, and publishes the report. I validate the framework in layers, starting with a simple test and adding abstractions only after they are proven useful."

Validate each component and be able to explain one execution path from test specification → fixture → page object → wrapper → assertion.

22. Suggested Practice Exercises

7. Create the project from an empty folder and run one sample Playwright test.
8. Implement the wrapper with fill, click, navigation and text retrieval methods.
9. Create the login page object and validate the login workflow.
10. Add a custom fixture for the login page and use it in a test.
11. Read account test data from CSV and create a record using a page object.
12. Generate unique data using Faker and verify the saved record.
13. Implement one API service method and test both success and error responses.
14. Enable trace/video/reporting and diagnose an intentionally failing test.
15. Run the project in GitHub Actions and retrieve the report artifact.
16. Recreate the architecture from memory and explain why each layer exists.

Appendix A. Key Commands

```
npm install
npx playwright install
npx playwright test
npx playwright test --headed
npx playwright test --debug
npx playwright test tests/TC001_create_account.spec.ts
npx playwright show-report
npx tsc --noEmit
```

Use npm install for initial setup or dependency changes. For reproducible CI installation, commit package-lock.json and use npm ci.

Appendix B. Implementation Order and Files

Implementation order	Source file(s)	Learning focus
1	package.json, tsconfig.json, playwright.config.ts	Dependencies, compiler options and runner settings.
2	helpers/playwright.ts	Reusable wrapper design and shared actions.
3	pages/selectors.ts and pages/salesforceLogin.ts	Selectors, inheritance and page-object methods.
4	pages/salesforceHomePage.ts and pages/salesforceAccountPage.ts	Navigation and business interactions.
5	customFixtures/salesForceFixture.ts	Fixture typing and object lifecycle.
6	data/account.interface.ts, data/accounts.csv, helpers/csvUtil.ts, helpers/fakerUtils.ts	Data modelling and parameterization.
7	tests/TC001_create_account.spec.ts	End-to-end orchestration and assertions.
8	api/api_services/* and api/api_Tests/*	Service-layer API automation.
9	helpers/*	Supporting utilities and their dependencies.
10	.github/workflows/playwright.yml	CI setup, execution and artifacts.